

Project Specification: Real-Time Voice AI Bot

1. Project Objective

Build a completely localized, real-time Conversational AI architecture for phone calls. The system acts as middleware connecting Asterisk telephony (via ARI) to local AI models (Whisper.cpp, local LLM, Piper TTS). The goal is to achieve sub-second conversational latency without relying on cloud APIs.

2. Technology Stack

- **Core Engine:** Node.js runtime with TypeScript.
- **Telephony Engine:** Asterisk 18/20+ (PJSIP) with `res_ari` and `res_ari_events` enabled.
- **ARI Client:** `ari-client` (npm package) for call control.
- **State Management:** Redis (for active channel tracking, call context, and lightweight queuing).
- **Database ORM:** Sequelize with MariaDB (for post-call records).
- **AI Microservices (Local):**
 - **STT:** Whisper.cpp (for fast audio-to-text inference).
 - **LLM:** Local Qwen or Llama 3 (served locally, text streaming).
 - **TTS:** Piper TTS (generating fast 16kHz `.wav` or raw PCM).

3. Architecture & Data Flow

Phase A: Call Ingestion & Media Forking

1. An inbound call hits the Asterisk dialplan and is routed to the custom Stasis application.
2. The Node.js app receives the `StasisStart` event.
3. The app answers the channel and creates an Asterisk mixing bridge.
4. The app creates an `ExternalMedia` channel via ARI, specifying the `slin16` format (16kHz raw PCM).
5. Asterisk connects the caller's audio stream via UDP/TCP to a socket listening on the Node.js application.

Phase B: The AI Processing Pipeline (The Loop)

1. **VAD (Voice Activity Detection):** The Node socket receives the `slin16` audio buffer. A lightweight VAD algorithm determines when the caller has stopped speaking.
2. **STT:** The buffered speech chunk is streamed to the local Whisper.cpp service. Whisper returns transcribed text.
3. **LLM:** The transcribed text is appended to the call's Redis conversation history. The updated context is sent to the local LLM, which streams its response back token-by-token.
4. **TTS:** As the LLM streams tokens, the Node app batches them into logical sentences (splitting by punctuation) and pipes them into Piper TTS.
5. **Playback:** Piper outputs raw 16kHz audio. The Node app immediately pipes this audio back into the Asterisk `ExternalMedia` socket to be played to the caller.

Phase C: Teardown & Post-Call Processing

1. The `StasisEnd` event fires when the caller hangs up.
2. The Node app destroys the bridges, closes the media sockets, and fetches the final conversation history from Redis.
3. The final transcript and call metrics (duration, timestamp) are saved to MariaDB using Sequelize.
4. The call session data is deleted from Redis.

4. Redis Data Model

Since RabbitMQ is excluded, Redis handles all state and transient data:

- `call:{channelId}:state` -> Current state of the call (e.g., "listening", "processing", "speaking").
- `call:{channelId}:history` -> Array or string containing the ongoing LLM chat history.
- `call:{channelId}:metrics` -> Hash storing start time, caller ID, etc.

5. Recommended Directory Structure

```
src/
├── config/
│   ├── asterisk.ts      # ARI connection credentials
│   ├── redis.ts         # Redis client initialization
│   └── database.ts      # Sequelize setup (MariaDB)
├── models/
│   └── CallRecord.ts    # Sequelize model definition
├── services/
│   ├── ari/
│   │   ├── stasis.ts   # ARI Stasis app logic & event listeners
│   │   └── media.ts     # UDP/TCP Socket server for Asterisk ExternalMedia
│   ├── ai/
│   │   ├── stt.ts      # Whisper.cpp HTTP/WebSocket client
│   │   ├── llm.ts       # LLM streaming client
│   │   └── tts.ts       # Piper TTS execution and buffering
│   └── utils/
│       ├── vad.ts       # Voice Activity Detection logic
│       ├── audio-utils.ts # PCM buffer manipulation
│       ├── logger.ts    # Logging configuration
│       └── index.ts     # App entry point
```

6. Implementation Instructions for the AI Assistant

When generating the code for this project, please adhere to the following:

1. **Strict TypeScript:** Use strict typing for all ARI events, Asterisk channel objects, and configuration variables.
2. **Node.js Streams:** Use `stream.Transform` and `stream.Readable` heavily when passing audio between Asterisk sockets, Whisper, and Piper. Do not store massive audio files in RAM.
3. **Non-Blocking Logic:** Do not run AI inference directly in the Node.js event loop. The `stt.ts`, `llm.ts`, and `tts.ts` files should act as lightweight clients communicating with external local servers.
4. **Graceful Teardown:** Ensure that when a call drops prematurely, all sockets are forcefully closed, and memory buffers are cleared to prevent memory leaks in production.

